Variants of Turing Machines (3.2)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

Turing Machine Variants: Enumerator

An **enumerator**, loosely defined, is a Turing machine with a printer. Instead of accepting input, it generates, in some order, all the strings accepted by its language.

Constructing an enumerator from a Turing machine *M* is easy, assuming the existence of the printer. Let s_1, s_2, … be an ordering of all the possible strings in the alphabet. Then:

On input string *w*, ignore it.

- Repeat for $i = 1, 2, \ldots$:
  - Repeat for $j = 1, \ldots, i$:
    - Run *M* for *i* steps on input s_j. If we accept, print s_j.

We enumerate in this odd breadth-first fashion instead of depth-first, to avoid spinning forever on a computation that doesn't halt. We will reuse this technique shortly.

Showing that some Turing machine *M* recognizes *A* if an enumerator *E* enumerates it is even easier.

On input string *w*:

- Run *E* modified so that every time *E* would print a string, it instead compares it with *w*.
- If they are ever the same, accept.

Turing Machine Variants: Enumerator

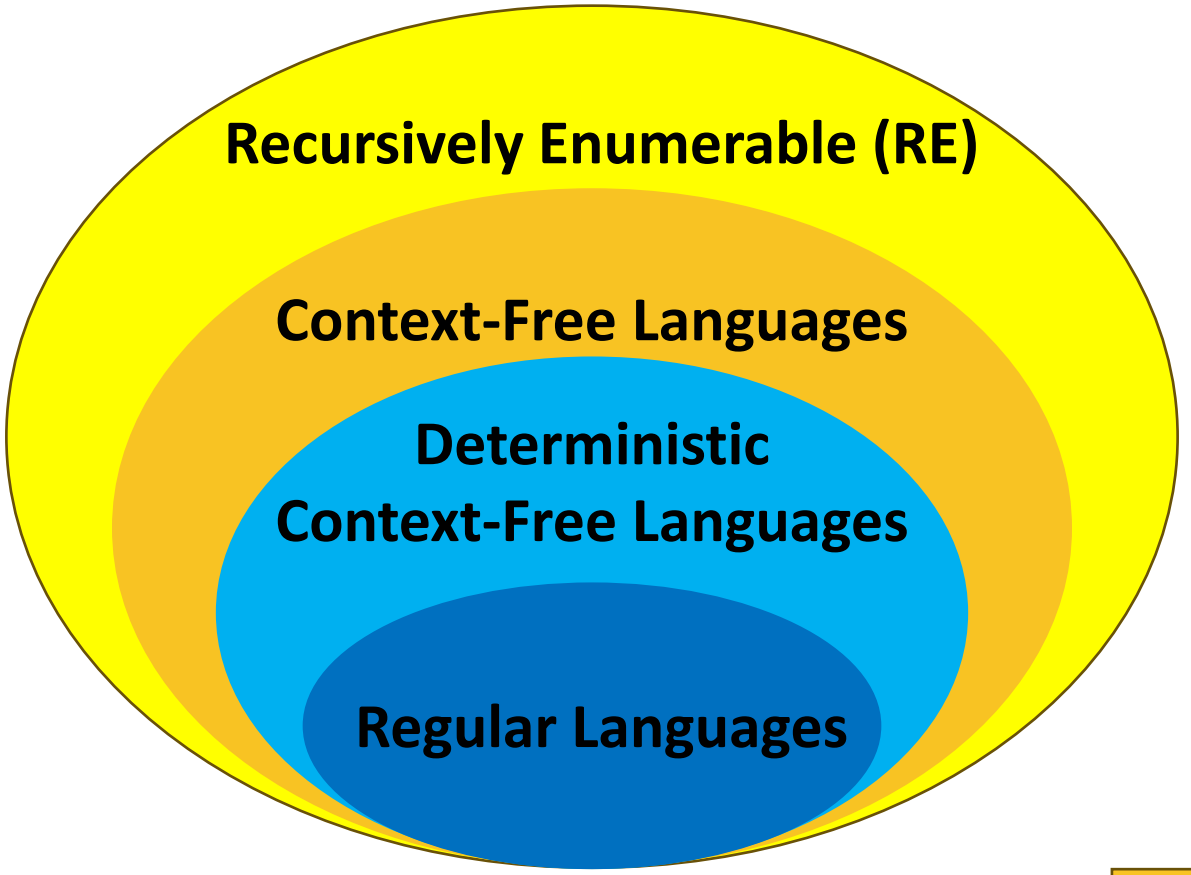
An **enumerator**, loosely … s odd breadth-first fashion with a printer. Instead of … t, to avoid spinning forever some order, all the string… at doesn't halt. We will … shortly.

Constructing an enumera… Turing machine *M* easy, assuming the exist… numerator *E* enumerates it is … be an ordering of all t… alphabet. Then:

If you've ever heard Turing-recognizable languages called **recursively enumerable** languages, this thing is why.

On input string *w*, igno…

- Repeat for $i = 1, 2, \ldots$:
  - Repeat for $j = 1, \ldots, i$:
    - Run *M* for *i* steps on input s_j. If we accept, print s_j.
- Run *E* modified so that every time *E* would print a string, it instead compares it with *w*.
- If they are ever the same, accept.



Don’t worry, there are still more categories!

Turing Machine Variants: Multitape TM

A **multitape Turing machine** has a finite number of tapes, each with its own read-write head.

- The input appears on tape 1, and the other tapes start out blank.
- On each transition we read all the tapes at once.
- On each transition we write to all the tapes at once.
- On each transition we move left or right on all the tapes at once.

So with *k* tapes, we say:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$$

Turing Machine Variants: Multitape TM

A **multitape Turing machine** has a finite number of tapes, each with its own read-write head.

- The input appears on tape 1, and the other tapes start out blank.
- On each transition we read all the tapes at once.
- On each transition we write to all the tapes at once.
- On each transition we move left or right on all the tapes at once.

So with *k* tapes, we say:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$$

We can simulate a multitape Turing machine with a single-tape Turing machine.

- Let # be a tape separator symbol.
- Add a "dotted" version of every tape alpbahet symbol to the tape alphabet.
- Store the multiple tapes one after the other on our single tape, with # between each one and dotted symbols indicating the locations of the heads.
- So the tape starts out as $\#\dot{w}_1 \ldots w_k \#\dot{\sqcup}\#\dot{\sqcup}\#\ldots\#$
- On each move, we:
  - Scan to find the # to indicate where each tape is, along with the dotted symbols
  - Simulate the movement of the respective head by changing the location of the dot
    - If we move right onto a #, we move *everything else on the tape one space to the right*
  - Simulate the other aspects of the transition as normal for the respective single-tape Turing machine

Turing Machine Variants: NDTM

A **nondeterministic Turing machine** is defined exactly the way you think it is.

$$\delta: Q \times \Gamma \rightarrow \mathbf{P}(Q \times \Gamma \times \{L, R\})$$

We can simulate one using a deterministic Turing machine.

- The key is to realize that *at any given time, there are a finite number of nondeterministic choices*.
- Now, think of the nondeterministic choices as a tree.
- We need to visit the tree **breadth first**, not depth first – otherwise we can spin forever on a branch that never stops computing.

Construct a three-tape Turing machine.

- The first tape remains the input as initially given.
- The second tape is where we actually execute.
- The third tape holds an address in a total order of nondeterministic choices.
  - Let b be the size of the largest set of possible nondeterministic choices on a given state and symbol. (Its upper bound is $|Q \times \Gamma \times \{L, R\}|$.)
  - Define a nondeterminism address as a string over $\{1, 2, \ldots, b\}$. Add all these symbols to Γ.
  - Order the addresses as:
    - 1..*b*
    - 11..1*b*..2*b*..*bb*
    - 111…11*b*…12*b*…*bbb*
    - …and so on.

Turing Machine Variants: NDTM

Construct a three-tape Turing machine.

- The first tape remains the input as initially given.
- The second tape is where we actually execute.
- The third tape holds an address in a total order of nondeterministic choices.
  - Let *b* be the size of the largest set of possible nondeterministic choices on a given state and symbol. (Its upper bound is $|Q \times \Gamma \times \{L, R\}|$.)
  - Define a nondeterminism address as a string over {1, 2, …, *b*}. Add all these symbols to Γ.
  - Order the addresses as:
    - 1..*b*
    - 11..1*b*..2*b*..*bb*
    - 111…11*b*…12*b*…*bbb*
    - …and so on.

On input *w*:

1. Initially, tape 1 contains *w*; 2 and 3 are empty.
2. Set tape 3 to 1.
3. Copy tape 1 to tape 2.
4. Use tape 2 to simulate the NDTM with *w* on the branch of its nondeterminism represented by tape 3.
   a. Before each step, consult the next symbol on tape 3.
   b. If no more symbols remain on tape 3, or the symbol value is larger than the number of choices we have, go on to step 5.
   c. If we reach a rejecting configuration, go to step 5.
   d. If we reach an accepting configuration, **accept and halt.**
5. Replace the string on tape 3 with the next string in the order. Go to step 3.

Turing Machine Variants: NDTM

Construct a three-tape T

- The first tape remains
- The second tape is wh
- The third tape holds ar
  nondeterministic choic
  - Let *b* be the size
    nondeterministic
    symbol. (Its upp
    $|Q \times \Gamma \times \{L, R\}|$.)
  - Define a nondeterminism address as a string over {1, 2, …, *b*}. Add all these symbols to Γ.
  - Order the addresses as:
    - 1..*b*
    - 11..1*b*..2*b*..*bb*
    - 111…11*b*…12*b*…*bbb*
    - …and so on.

2 and 3 are empty.

NDTM with *w* on the branch of
nted by tape 3.

t the next symbol on tape 3.

in on tape 3, or the symbol
value is larger than the number of choices we have, go on to step 5.

c. If we reach a rejecting configuration, go to step 5.

d. If we reach an accepting configuration, **accept and halt.**

5. Replace the string on tape 3 with the next string in the order. Go to step 3.

This simulates recognition.
Decision is a little trickier.

You'll cover that later.

Turing Machine Capabilities

Turing Machine Power

- We've come to yet another interesting point in our discussion of Turing machines.
- For context-free languages, unlike for regular languages, non-determinism actually adds power. Recalling more completely, NFAs have the same computational power as DFAs. DPDAs are more powerful than either, and NPDAs are more powerful than DPDAs.
- Given that Turing machines are even more powerful than NPDAs it would seem intuitive that the same would be true for them.
- It is intuitive. It is also wrong, because of something both very special about Turing machines, and not special at all.

Turing Machine Power

- We've come to yet another interesting point in our discussion of Turing machines.
- For context-free languages, unlike for regular languages, non-determinism actually adds power. Recalling more completely, NFAs have the same computational power as DFAs. DPDAs are more powerful than either, and NPDAs are more powerful than DPDAs.
- Given that Turing machines are even more powerful than NPDAs it would seem intuitive that the same would be true for them.
- It is intuitive. It is also wrong, because of something both very special about Turing machines, and not special at all.

Computationally, nothing is more powerful than a Turing Machine.

Turing Machine Power

- We've come to yet another interesting point in our discussion of Turing machines.
- For context-free languages, unlike for regular languages, non-determinism actually adds power. Recalling more completely, NFAs have the same computational power as DFAs. DPDAs are more powerful than either, and NPDAs are more powerful than DPDAs.
- Given that Turing machines are even more powerful than NPDAs it would seem intuitive that the same would be true for them.
- It is intuitive. It is also wrong, because of something both very special about Turing machines, and not special at all.

Computationally, nothing is more powerful than a Turing Machine.

- But we can go farther than that, and now, we will.

Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013